

Week 12 deep learning

1. What is deep learning?

Deep learning is a sub-class of neural nets. Normally has two or more hidden layers.

2. What is deep feed-forward neural network and how does it work?

Deep FFNN is the same as FFNN but with more hidden layers. The number of nodes in each layer may vary, and edges from each node in one layer is connected to each node in the next layer.

It has one matrix of weights for each layer. It uses the same activation function for all hidden layers

3. How can we update in deep FFNN?

Compare output value to target value and calculate loss and update if loss is more than 0. All weights are updated according to their contribution to the loss.

$$w_{i,j}^{(k)} = w_{i,j}^{(k)} - \eta \frac{\partial}{\partial w_{i,j}^{(k)}} L(t, y)$$

(eventually:)

$$w_{i,j}^{(k)} = w_{i,j}^{(k)} - \eta \delta_j^{(k)} a_i^{(k-1)}$$

4. What is vanishing gradient problem?

The vanishing gradient problem is a challenge that can occur during the training of deep neural networks. During backpropagation, the gradient of the loss function with respect to the weights of the neural network is computed, and these gradients are used to update the weights via gradient descent. However, in deep neural networks, as the gradient is propagated backwards through the network, it can become smaller and smaller, eventually becoming very close to zero.

When the gradient becomes very small, it can cause the weights of the earlier layers to be updated very slowly, which can lead to slow or even stagnant learning in those layers. This can make it difficult for the network to learn complex representations of the input data, and can make it more difficult to train deep neural networks with many layers.

The vanishing gradient problem can occur in particular types of activation functions, such as the sigmoid function, which can saturate to zero or one for large or small inputs, respectively. This can cause the gradients to become very small as they propagate backwards through the network.

5. What does it mean that the sigmoid function can saturate? What problems can this cause?

the sigmoid function, "saturate" refers to a situation where the output of the function approaches its maximum or minimum limit as the input value becomes very large or very small.

For example, the sigmoid function has an output range between 0 and 1. If the input to the sigmoid function is very large (positive or negative), the output of the function will approach 1 or 0, respectively, and stay there even if the input increases further. This means that the sigmoid function's output is "saturated" at the upper or lower bounds of its output range.

6. What is ReLU?

the rectified linear unit (ReLU) function, can help to alleviate the vanishing gradient problem, as they do not saturate in the same way as sigmoid functions.

- $ReLU(x) = \max(x, 0)$
- $ReLU'(x) = 1$ for $x > 0$
- $ReLU'(x) = 0$ for $x < 0$
- Use 0 for $ReLU'(0)$

7. What is convolutional NNs?

A Convolutional Neural Network (CNN) is a type of neural network commonly used for image processing, computer vision, and other tasks that involve analyzing spatially-structured data.

CNNs are designed to automatically and adaptively learn spatial hierarchies of features from input images, by applying a sequence of convolutional, pooling, and nonlinear activation layers.

In a typical CNN, the input image is processed through a series of convolutional layers, which apply a set of filters to the input image to extract low-level features such as edges and corners. The output of the convolutional layers is then passed through a pooling layer, which downsamples the feature maps to reduce their size and computational complexity. Finally, the pooled feature maps are fed into a fully connected layer or layers, which perform classification or regression on the final output.

Vi har filter, som vi definerer selv eks 3x3. Også har vi bildet med pixler. For å gjenkjenne bildet bruker vi 3x3 filteret også går vi over hver eneste 3x3 blokk med pixler. Også tar vi dot-produkt eller matrise multiplikasjon. Tilslutt får vi feature map av bildet, som er vår representasjon av bildet. Også sender vi det til neste lag. Da kan vi gjøre pooling. Hvert filter detekterer pattern.

I de første lagene detekterer vi enkle features (eks kanter). Jo dypere i nettet, desto mer avanserte features detekterer/finner man.

Pooling reduce size of data to extract features.

8. What is recurrent neural nets?

Recurrent Neural Networks (RNNs) are a type of neural network commonly used for processing **sequential data**, such as time series, speech, and natural language processing.

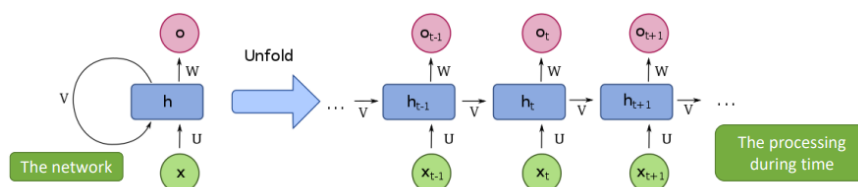
Unlike feedforward neural networks, which process input data in a single pass through the network

RNNs have loops within the network architecture that allow information to persist over time. This allows them to capture temporal dependencies and long-term contextual information in the input data.

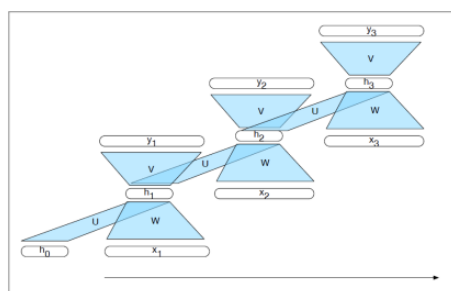
In an RNN, each neuron in the network receives input not only from the previous layer but also from its own output at the previous time step. This creates a feedback loop within the network, allowing the network to maintain a hidden state that can represent the context of previous inputs.

The output of each neuron in an RNN depends not only on the current input but also on the network's hidden state, which carries information about the context of previous inputs. This hidden state is updated at each time step and serves as a memory that the network can use to make predictions or classifications.

h_t in the cell at time t is determined by input x_t at time t and the state h_{t-1} at time $t-1$



Forwards



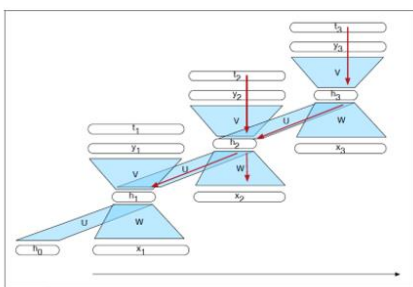
From J&M, 3.ed., 2019

- $x_1, x_2, \dots, x_n$ is the input sequence
- Each U, V and W are edges with weights (matrices)
- Forward:
 1. Calculate h_1 from h_0 and x_1 .
 2. Calculate y_1 from h_1 .
 3. Calculate h_i from h_{i-1} and x_i , and y_i from h_i , for $i = 1, \dots, n$

- The same U, W , and V for all layers
- $\mathbf{h}_t = g(\mathbf{h}_{t-1}U + \mathbf{x}_tW)$
- $\mathbf{y}_t = f(\mathbf{h}_tV)$
- g and f are activation functions
- f is often softmax, i.e.,
 - $\mathbf{y}_t = \text{softmax}(\mathbf{h}_tV)$
- (There are also bias terms which we didn't include in the formulas)

A recurrent neural network (RNN) is a network which contains a cycle within its network connections. It may be a connection from a node to itself. It could also be a cycle including more nodes. The RNN is processed in time steps. The value of a node x at time $t+1$ is calculated from the values of all the nodes at time t and the direct connections into node x . In particular, if there is a connection from a node x into itself, the state of x at time $t+1$ will be influenced by the state of x at time t (and all earlier states). This gives the network a form of memory. An RNN is particularly suited for sequence problems, for example sentences in natural language, where it reads one word at a time and “remembers” the earlier words in the sentence

Training



From J&M, 3.ed., 2019

- At each output node:
 - Calculate the loss and the δ -terms
- Back-propagate the error, e.g.
 - the δ -terms at h_2 are calculated
 - from the δ -terms at h_3 by U and
 - the δ -terms at y_2 by V
- Update
 - V from the δ -terms at all the y_i -s
 - U, W from the δ -terms at all the h_i -s